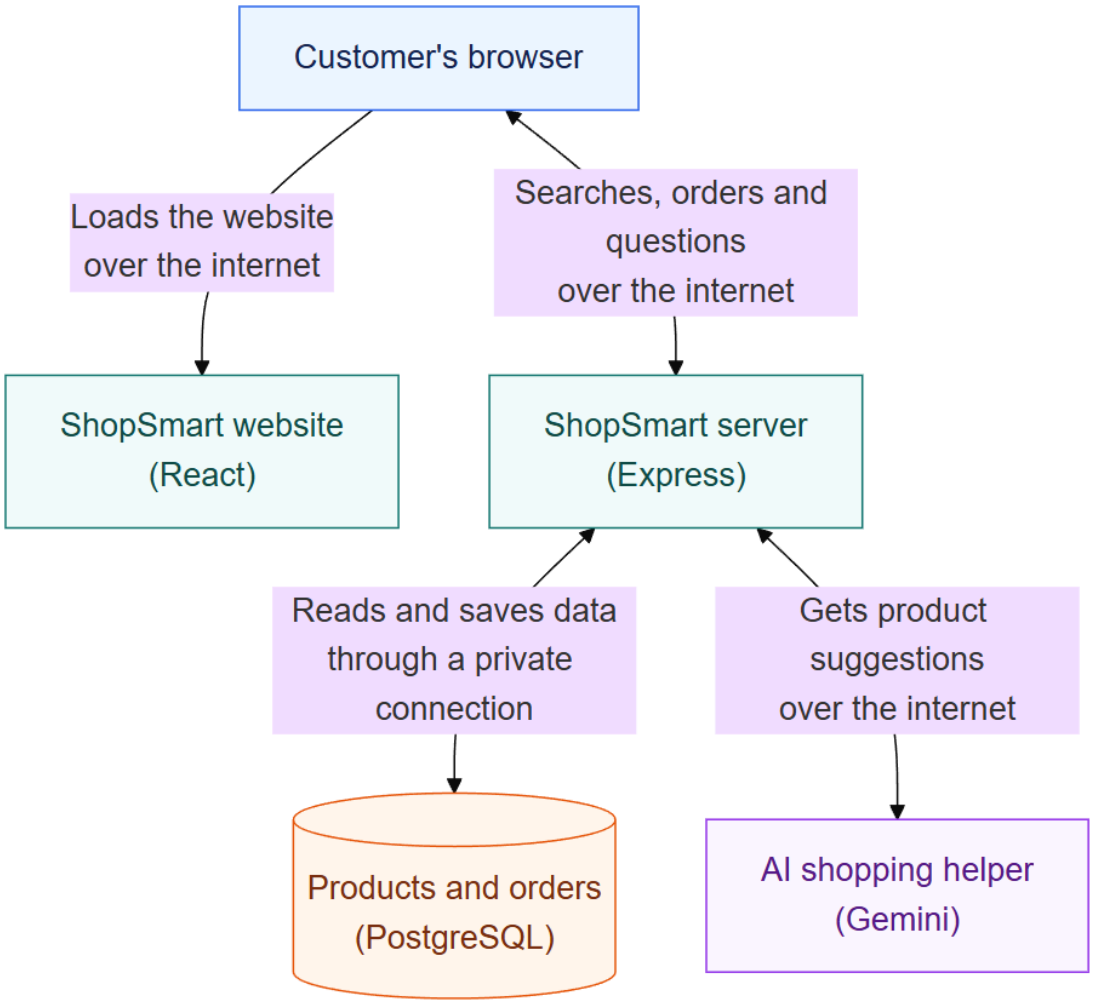
ShopSmart deployment plan - individual release

Author: Paul Richard Calma
Date: 12 November 2026 (Asia/Manila)
Status: local release candidate only. No hosting approval, staging URL or access gateway was supplied. The domains below are planning placeholders and have not been deployed.

Open the editable Mermaid architecture source. The **.mmd** file is the primary diagram artifact. Edit it in a Mermaid-capable editor; the same source is embedded below for Markdown preview.

View the rendered preview.

How ShopSmart connects



The customer opens ShopSmart in their browser. The website provides the screens and buttons. When the customer searches, places an order or asks a question, the browser sends that request to the ShopSmart server. The server reads products and saves orders in the database. For shopping questions, it can ask Gemini for help choosing products; if Gemini is unavailable, it answers using the product catalog.

The arrows show the planned hosted setup. **For the current local release, the website, server and database run on the same computer. Only Gemini needs an internet connection.** No staging site has been deployed.

Where the settings belong

Part	Settings it needs	Who can see them?
Website	The server's web address, called **VITE_API_URL**	Anyone using the website; it is not a secret.
Server	The allowed website address (**FRONTEND_URL**), database connection (**DATABASE_URL**) and optional Gemini key (**GEMINI_API_KEY**)	Keep the database connection and Gemini key private on the server.
Database	Its own account, password and saved product/order data	Only the server and the person managing the database should have access.

Only product information and the shopping question go to Gemini. Checkout names, email addresses and delivery addresses do not. Use fictional customer details for this activity.

Service and environment settings

Service	Planned runtime and URL	Configuration and secret boundary
Browser + React	An instructor-approved static host; proposed **https://shopsmart-staging.example**	Build from repository root with **npm ci**, then **npm run build**; publish **frontend/dist**. Set **VITE_API_URL=https://shopsmart-api-staging.example/api** at **build time**. It is public. Configure SPA fallback to **index.html** for **/cart**, **/checkout**, **/assistant** and **/products/:id**.
Express API	Instructor-approved private Node service; proposed **https://shopsmart-api-staging.example**; health at **https://shopsmart-api-staging.example/api/health**	Install from repository root, run **npm run db:generate**, then start with **npm start --workspace backend** (runs with backend as working directory). Set platform-assigned **PORT**, **NODE_ENV=production**, **FRONTEND_URL=https://shopsmart-staging.example**. Store **DATABASE_URL**, optional **GEMINI_API_KEY**, and selected **GEMINI_MODEL** in the host's server secret/settings manager.
PostgreSQL	Approved managed PostgreSQL instance on a private network	Only API and controlled migration jobs receive database access. Use TLS and the provider's required connection settings. Keep persistent storage and backup/restore capability. Do not put a database URL/password in browser configuration, logs, screenshots, or Git.
Gemini	Existing optional external provider integration	API server calls Gemini over HTTPS using its server-side key. Choose an approved model available to the account. An unavailable/missing provider produces a catalog rules answer. An external request has an 8-second timeout. No provider key goes to the browser or static host build.

FRONTEND_URL permits exactly the planned frontend origin in the existing CORS configuration. Rebuild the frontend after changing its public API URL; restart the API after server environment changes. CORS is a browser policy, **not authentication or access control**. The current health endpoint checks that Express responds; confirm products/orders separately to establish database readiness.

Non-destructive database release sequence

1. Preserve an existing database with a host-managed backup before any schema release. Use a separate fictional-data staging database.
2. Commit all reviewed migrations. Restore the canonical initial migration directory **20260910093853_initial_schema**: its LF SQL SHA-256 matches this local database's stored applied checksum. Do not alter the applied database migration record.
3. In the approved release job, install dependencies, generate Prisma Client (**npm run db:generate**), then run **npm run db:deploy**. This executes Prisma 6 **migrate deploy**, not **migrate dev**. It neither seeds sample products nor resets orders.
4. If a different evaluator database has history conflicts, stop and compare its migration history/schema against the submitted SQL. Do not reset it, blindly mark a migration applied, or copy this machine's history records into it. A fresh empty evaluator database can apply the committed migration normally.
5. For an explicitly empty disposable classroom database only, run **npm run db:seed** once to supply fictional products. This script deletes **all** orders, order items and products; never use it as routine startup or a deployment hook.
6. Start/restart the API after migration success, publish the static build, then verify health, products, cart, one fictional checkout, stock rejection, assistant sources and provider failure fallback.
7. Record order IDs and stock before/after verification. Never reseed at the end. Retain backups; roll the application back to a compatible build if a release fails, and review data migration recovery separately.

Staging access and verification gate

Because authentication and roles are absent, keep the current release local. Before approved staging, place **both** the static frontend and API behind an instructor-approved identity gateway, VPN or equivalent access restriction for Paul Richard Calma and the evaluator. A private frontend alone leaves a public API reachable. Do not expose a public checkout that collects real data.

Once hosting, database and access restrictions are approved, record actual URLs (replace all **.example** placeholders), authorized evaluator access instructions without credentials, deployment date/time in Asia/Manila, submitted version, and expected/observed results for the same smoke cases. Confirm denied access from an unauthenticated client to both services. Use only Alexandra Eala / **alexandra.eala@example.test** / **123 Fictional Street, Demo City** or equivalent fictional values. Staging smoke result is currently **not run**.

Hosting-specific commands are intentionally not invented before a platform is approved. Verify that platform's current build, SPA routing, secret manager, networking and access-gateway documentation at deployment time.

Verified references

- Prisma ORM 6 development and production migration workflow: **migrate deploy** applies committed migrations for production/testing; client generation is separate; it does not detect all schema drift.
- Vite environment variables and modes: **VITE_** variables are exposed in browser builds and must not contain secrets.

Both references checked on 12 November 2026.